Lab 3: Architecture Justification

Airport Lost Luggage Claim & Tracking Portal

Architecture Selection Component Diagram

I have chosen Microservices Architecture (with an API Gateway) for the Airport Lost Luggage Claim & Tracking Portal.

Microservices architecture: Independent scaling, fault isolation and replaceable components behind stable interfaces — matches NFR-001 and NFR-002.

Architectural Choice: Two Reasons

- Independent scaling for the performance requirement. NFR-001 needs the full multi-terminal scan history in under 200 ms at peak terminal traffic, while claim registration and compensation are low-volume. The Bag Tracking Service (with its own indexed scan-log store) can scale horizontally without scaling the rest of the system.
- Fault isolation around external systems. The portal depends on the Airline Booking System (FR-004) and the Payment Gateway (FR-005). Each is reached only through one service (Claim Management and Compensation respectively), so a slow or failed external call cannot bring down baggage tracking, supporting the 99.9% uptime target (NFR-002); a provider can also be swapped without touching other components because the interface stays the same.

Security Advantage

All traffic enters through the API Gateway, which authenticates users (agent-only timeline and compensation access, FR-003) and terminates TLS 1.2+. Internal services are never exposed directly, service-to-service calls run over TLS, and passenger PII lives only in the Claim Management Service's data store, encrypted with AES-256 at rest (NFR-002). A compromise of one small service therefore exposes far less than in a single shared codebase.

Performance Benefit

The Bag Tracking Service reads from a scan-log index keyed by bag tag number and can be cached and scaled on its own, keeping timeline queries under 200 ms during peak load. Heavy scan matching runs in that service rather than in the claim flow, so claim submission can still return the Claim ID within 5 seconds (FR-002).

Trade-off Accepted

Microservices add network latency and operational complexity. We accept this because the gateway and containerised deployment contain it, and the scalability and availability gains matter more for a 24/7 airport system.